Technical Interview - Boilerplate 1 - Node + Typescript + PostgreSQL

Giuliana Olmos · 31 de octubre de 2024 · https://dev.to/giulianaolmos/technical-interview-boilerplate-1-node-typescript-postgresql-an1

Introduction

Hi! Hello! 🤗

You might have been expecting another post about technical interview questions (don't worry, I'll share more of those soon!). But today, I want to talk a bit about my experience with interviews and share some tips that worked for me.

I've been doing interviews for a few months now, mostly for Senior Backend Developer roles. If I had the chance, I aimed for positions as a Founding Backend Engineer. In many of these interviews, I was given take-home assignments. If not, they usually asked me to build a project from scratch—covering things like deployment, CI/CD, architecture, servers, databases, etc.

That's why I decided to write this post.

After doing several interviews, I realized that the tasks and questions tend to repeat. That's when I knew I needed to find a way to improve my process.

For me, creating reusable boilerplates was a game-changer. Having a solid base to start with helped me focus on the important parts, like the business logic, rather than starting from scratch each time.

In this post, I'll talk about one of the first boilerplates I made—a template for projects that require a ✨server✨(using TypeScript, Node.js, and Express) and a ✨relational database✨(PostgreSQL).

I won't share all the code here, but I'll show you some key parts and explain why I included them.

To use this boilerplate, you'll need to have

- Node.js,
- PostgreSQL
- and Docker,

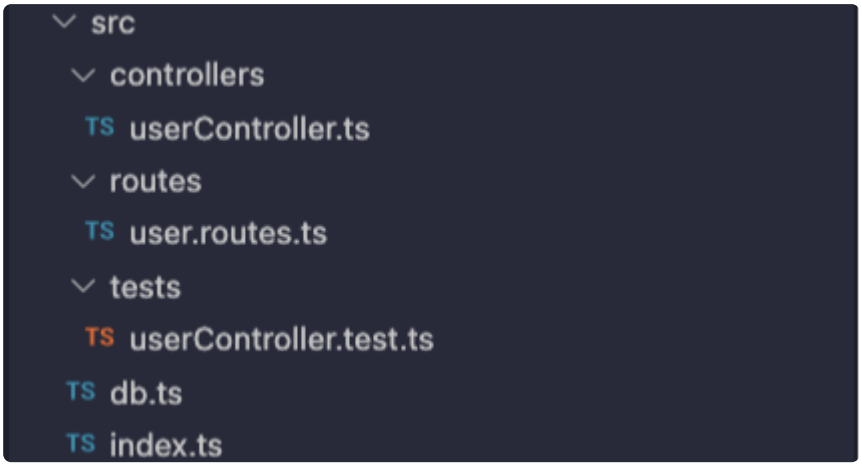
installed.

Topics

```
* [Server](#topic-1)
* [Database](#topic-2)
* [Docker](#topic-3)
* [Docker Compose](#topic-4)
* [How to run the project?](#topic-5)
* [How to stop the project?](#topic-6)
* [Tests](#topic-7)
```

Server:

The project in the boilerplate is really simple—just a basic CRUD. The idea is to have a base to start working from.



Initialize the project using NodeJs, Typescript, and Express.

```
npm init -y
```

And then install some dependencies.

```
npm install express
npm install --save-dev typescript ts-node @types/node @types/express
```

My `tsconfig.ts` looks like this.

```
"compilerOptions": {
  "target": "ES2020",
  "module": "commonjs",
  "strict": true,
  "esModuleInterop": true,
  "skipLibCheck": true,
```

```
    "forceConsistentCasingInFileNames": true,
    "outDir": "./dist"
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

I created some basic endpoints as examples of each type. The routes are located in `user.routes.ts`.

```
import { Router } from "express";
import UserController from "../controllers/userController";


const userRoutes = Router();
const userController = new UserController();


userRoutes.get("/users", userController.getIndex);
userRoutes.post("/users", userController.createUser);
userRoutes.post("/usersTransaction", userController.createUserTransaction);
userRoutes.get("/users/:id", userController.getUserById);
userRoutes.put("/users/:id", userController.updateUser);
userRoutes.delete("/users/:id", userController.deleteUser);


export default userRoutes;
```

I also added `userRoutes.post("/usersTransaction", userController.createUserTransaction);` to include an example of a transaction using PostgreSQL.

Until now, I've never had to use a transaction query in a take-home project 🥹, but I think it's a good idea to have it implemented in this boilerplate in case I need it in the future. 🤓

Database:

For this boilerplate, I decided to use ✨PostgreSQL✨ instead of MySQL. The reason is that most of the positions I apply for expect experience with PostgreSQL.

I installed it using:

```
npm install pg
```

And created a pool in `db.ts.`

```
import { Pool } from "pg";

const pool = new Pool({
```

```
  user: "myuser",
  host: "db",
  database: "mydb",
  password: "mypassword",
  port: 5432,
});

export default pool;
```

The data for my local database is set in my `docker-compose.yml` file.

A good practice (especially if you plan to deploy this) is to store all the connection details in an `.env` file.

Once the database connection was set up, I completed the controllers with some queries. I created a CRUD with a few actions to provide examples of each query type.

To avoid spending too much time researching later, I included a link to the PostgreSQL query documentation as a "cheat sheet" in the boilerplate for future reference.

```
// https://node-postgres.com/features/queries
class UserController {
  public async getIndex(req: Request, res: Response): Promise<void> {
    try {
      const result = await pool.query("SELECT * FROM users");

      console.log({ result: JSON.stringify(result) });

      res.status(200).json(result.rows);
    } catch (error) {
      console.error(error);
      res.status(500).json({ error: (error as Error).message });
    }
  }
}
```

I also implemented a transaction and linked to the relevant documentation. While a transaction isn't necessary for this particular case, I wanted to include everything I might need in a take-home project to save time in the future. It also helped me understand how transactions work and what additional code is required.

For example, in this case, you need to create a "client" for the transaction where you can perform actions like `BEGIN`, `COMMIT`, and `ROLLBACK`.

The documentation states:

> You must use the same client instance for all statements within a transaction. PostgreSQL isolates a transaction to individual clients. This means if you initialize or use transactions with the pool.query method, you will encounter problems. Do not use transactions with the pool.query method.

```
//https://node-postgres.com/features/transactions
 public async createUserTransaction(
   req: Request,
   res: Response
 ): Promise<void> {
   const client = await pool.connect();

   try {
     await client.query("BEGIN");
     const users = req.body.users;
     const promises: Promise<QueryResult<any>>[] = [];
     for (const user of users) {
       const { name, email } = user;
       promises.push(
         client.query("INSERT INTO users (name, email) VALUES ($1, $2)", [
           name,
           email,
         ])
       );
     }

     await Promise.all(promises);

     await client.query("COMMIT");
     res.status(200).json({ message: "Users added successfully" });
   } catch (error) {
     console.error(error);
     await client.query("ROLLBACK");
     res.status(500).json({ error: (error as Error).message });
   }
 }
```

Docker:

I wanted to provide the option to run the server project locally. Sometimes, take-home projects don’t require a database. In those cases, you can remove the unnecessary code and simply run the server using Docker.

Here’s what my `Dockerfile` looks like:

```
FROM node:18-alpine

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

EXPOSE 3000
```

```
CMD ["npm", "run", "serve"]
```

Here’s how my Dockerfile is set up:

- `FROM node:18-alpine` — This is the base image I’m using.
- `WORKDIR /usr/src/app` — It sets the working directory inside the container.
- `COPY package*.json ./` — Copies `package.json` and `package-lock.json` into the working directory.
- `RUN npm install` — Installs the necessary dependencies.
- `COPY . .` — Copies the rest of the application code. (Make sure to use two dots! I once used just one and spent a while figuring out how to fix it 😄)
- `RUN npm run build` — Builds the TypeScript code.
- `EXPOSE 3000` — Exposes port 3000 so the app can be accessed.
- `CMD ["npm", "run", "serve"]` — This runs the app. (This line comes from your `package.json` file.)

```
"scripts": {
   "start": "ts-node src/index.ts",
   "build": "tsc",
   "serve": "node dist/index.js",
 },
```

Docker Compose:

Docker Compose will be your best ally if your take-home project requires a server and a database.

Why? 🤓☝️ Docker Compose allows you to run your server and database locally and ensures they are connected to each other.

I think this is a good approach when a take-home project requires both a server and a relational database. Hosting this kind of application live can be expensive, especially since relational databases (which usually come with costs on most cloud services) are involved. And for a take-home project, I don't recommend spending money to keep them live.

Been there, done that 😄

So, using Docker Compose is a great option to run the entire project on your computer, and it's also really easy to set up.

My `docker-compose.yml`

```
version: '3'
```

```yaml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    ports:
      - "5433:5432"
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./sql:/docker-entrypoint-initdb.d
    container_name: my_postgres_db

  app:
    build:
      context: .
    environment:
      DATABASE_URL: postgres://myuser:mypassword@db:5432/mydb
    ports:
      - "3000:3000"
    depends_on:
      - db
    container_name: my_express_app

volumes:
  db-data:
```

I set up two services in my `docker-compose.yml` file:

1. **Database (db):** In the `environment` section, I set the necessary data to connect to my database. To avoid exposing sensitive information, you can use a `.env` file.
2. **Server:** This service is built using my existing Dockerfile and connects to the database that I defined in the same Docker Compose setup.

Here are some common questions I want to address:

1. *If I have a docker-compose, do I still need a Dockerfile?*

Yes, you still need a Dockerfile if you are building a custom image for your application in a multi-service setup like with Docker Compose. The Dockerfile defines how to build the image for your Node.js app, while `docker-compose.yml` orchestrates how the services (like your Node.js app and PostgreSQL) run together.

For example, when you write:

```yaml
build:
  context: .
```

That means that you are using the Dockerfile from the current directory to build the server.

2. *I run my docker-compose, and the container is created with random and funny names. Hoy I could change that?*

Sometimes, when you're creating your `docker-compose` file from a template, you might forget to set a name for your containers. This will result in Docker assigning random, funny names like “beautiful_rainbow_container.”

To avoid this and be more specific, don’t forget to set a proper name for your service using the `container_name` option like this:

```
        container_name: my_postgres_db
```

This way, your container will have a meaningful and predictable name.

3. *I need to start my database with data already inserted. How can I do that?*

If your take-home project needs to be tested with pre-existing data, you’ll need to create a database that already contains this data from the start. To achieve this, you can run a script to insert the data during the Docker Compose build, every time you set the volume for your database.

Here’s how to do it:

In the `volumes` section of your database service, add the following lines:

```
        volumes:
          - db-data:/var/lib/postgresql/data
          - ./sql:/docker-entrypoint-initdb.d
```

- The first line defines the volume for your database: `db-data:/var/lib/postgresql/data.`
- The second line mounts a local directory (`./sql`) to the container’s initialization script directory (`/docker-entrypoint-initdb.d`).

This setup ensures that when you run docker-compose, it will look for the `sql` folder and execute all the scripts inside.

To ensure everything runs in the correct order, make sure your SQL scripts are sorted properly. For example, you might have:



This process ensures that your database is populated with the necessary data when the container is built.

4. *If I stop (down) the project, will I lose my data?*

No, you won't lose your data. All the data is stored in the volume called `db-data`, which is declared in the `docker-compose.yml` file. So, even if you stop the project (`docker-compose down`), the data you've been working on will remain intact unless you manually delete the volume.

When you run `docker-compose up` again, the database will be restored from that volume.

How to run the project?

To build and **run the entire project** locally, you can use the following command:

```
        docker-compose up
```

This command starts and runs all the services defined in your `docker-compose.yml` file. It also sets up the necessary containers, networks, and volumes. If the images haven't been built, it will automatically build them before starting the containers. This command ensures that your entire project is up and running.

How to stop the project?

If you want to **stop the Docker Compose services**, you have two options:

- `docker-compose down` — This stops and removes the containers but keeps the volumes, so your data remains intact.
- `docker-compose down -v` — This stops the containers and removes all named volumes declared in the `docker-compose.yml` file. Be careful with this option, as it will delete any data stored in those volumes.

Tests

To test this project, I use ✨Jest✨. I've set up a simple test suite to test the CRUD functionality. It's not complicated, but it's a solid base for my boilerplate.

Install the testing framework:

```
        npm install --save-dev jest ts-jest @types/jest
```

Set up the configuration:

```
npx ts-jest config:init
```

This will create the `jest.config.js` file for you.

```
/** @type {import('ts-jest').JestConfigWithTsJest} **/
module.exports = {
  testEnvironment: "node",
  transform: {
    "^.+.tsx?$": ["ts-jest",{}],
  },
};
```

Write your tests, add a line in `scripts` in `package.json`

```
"test": "jest"
```

And then, run:

```
npm i
npm run test
```

The end

Now that you have a boilerplate, the next time you need to complete a take-home project, all you need to do is clone the repo, rename it, and start working on your solutioN. ✨No need to worry about the initial configuration!✨

I haven't included all the code in this post, so if you want to check it out, here's the repo: https://github.com/GiulianaEOlmos/boilerplate-node-express-postgresql

Feel free to use it for your interviews, but I encourage you to create your own boilerplate with all the things you think you'll need. That way, you'll learn how to set it up and be ready to explain everything to your interviewers.

I hope this post was helpful to everyone. If you have any suggestions or improvements, please let me know in the comments.

And if any part of this post helped you in your job search, I'd love to hear about it!

Thank you so much, and have a great week!